



**SCHOOL OF SCIENCE &
ENGINEERING SPRING 2026**

Milestone 4:
Model Development and Evaluation

**AI-Based Story Point Estimation for
Agile Software Development**

Repository: <https://github.com/amiraHadimi/CSC5382-AI-Digital-Transformation-Project/tree/milestone4-final/Milestone%204>

REALIZED BY:
AMIRA HADIMI

SUPERVISED BY:
DR. ASMAA MOURHIR

1. Introduction

Milestone 4 represents the model development and evaluation phase of the AI-Based Story Point Estimation project. Building upon the data acquisition and preprocessing work completed in previous milestones, this phase focuses on integrating a complete, reproducible, and monitored machine learning system using industry-standard MLOps tools.

The central objective of this milestone is not merely to evaluate a predictive model, but to demonstrate a rigorous end-to-end MLOps pipeline that satisfies the requirements of modern AI engineering: modular architecture, experiment tracking, reproducibility, and environmental monitoring. The project applies this pipeline to the problem of automatically estimating story points in Agile software development contexts, using issue data sourced from 16 real-world JIRA projects.

This report documents all four core requirements of Milestone 4, namely: project structure and modularity, code versioning, experiment tracking and model versioning with MLflow, and MLOps platform integration via ZenML. An optional requirement regarding energy efficiency measurement using CodeCarbon is also addressed.

2. Project Overview and Model Strategy

The goal of this project is to automate the estimation of story points – a common Agile measure of task complexity – using natural language processing applied to JIRA issue descriptions and titles. Accurate story point estimation is a persistent challenge for development teams, and AI-based approaches offer the potential to reduce human bias and improve consistency across planning cycles.

2.1 Model Strategy

This milestone employs a dual-model strategy, combining an industrial-scale pretrained model with a lightweight baseline to balance evaluation quality and training demonstration:

- **Llama3SP (Main Evaluation Model):** An industrial-scale large language model fine-tuned specifically for Agile story point estimation. Llama3SP is based on the Llama 3.2-1B architecture and is loaded with project-specific LoRA adapters for each of the 16 JIRA projects. It is used exclusively for offline evaluation, as per real-world MLOps practices where pretrained models are reused and evaluated rather than retrained from scratch.
- **TF-IDF + Linear Regression (Baseline Model):** A lightweight baseline model trained within the pipeline to satisfy the training capability requirement. This model vectorises issue text using TF-

IDF and applies linear regression for story point prediction. It is integrated into the pipeline via the `train_tracking_step`.

This design choice is intentional and reflects contemporary MLOps practice: the primary contribution of the milestone is the system built around the model, including tracking, reproducibility, and monitoring infrastructure.

3. Project Structure and Modularity

The Milestone 4 codebase is organized following the Cookiecutter Data Science project template, adapted for an MLOps context. This structure ensures separation of concerns, maintainability, and ease of onboarding for collaborators.

3.1 Directory Layout

The repository is organized into the following top-level directories:

- `src/`: Contains all core source code, organized into four subpackages: `pipeline/` (model loading, inference, ZenML step and pipeline definitions), `evaluation/` (metrics computation), `tracking/` (MLflow and CodeCarbon integration wrappers), and `utils/` (configuration helpers).
- `configs/`: Houses the centralized parameter file `params.yaml`, which controls all configurable hyperparameters and pipeline settings.
- `tests/`: Contains unit tests for the metrics module and configuration loader, following standard testing practices.
- `results/`: Generated at runtime (excluded from version control), containing per-project prediction CSVs, aggregate metrics, and carbon reports.

3.2 Modularity Principles

Each concern in the pipeline is isolated in its own dedicated module. Model loading logic resides in `model_loader.py`, inference in `inference.py`, metrics computation in `metrics.py`, MLflow integration in `mlflow_tracker.py`, and environmental monitoring in `carbon_tracker.py`. This ensures that pipeline orchestration steps — defined in `zenml_steps.py` and `run_pipeline.py` — act as thin coordinators that delegate all domain logic to the appropriate modules, minimizing coupling and maximizing reusability.

All packages expose clean `__init__.py` interfaces, and the configuration system ensures that pipeline behavior can be modified exclusively through `params.yaml` without touching source code.

4. Code Versioning

Version control for this project is managed using Git and hosted on GitHub. The development workflow follows a milestone-based branching strategy, with the branch milestone4-final representing the stable, deliverable state of this phase.

The focus of the versioning strategy is on maintaining a clean, traceable, and reproducible codebase. Commit history documents the progressive implementation of pipeline components, from initial directory scaffolding through to full pipeline execution and validation. This approach supports auditability and enables collaborators to roll back to any prior state of the pipeline.

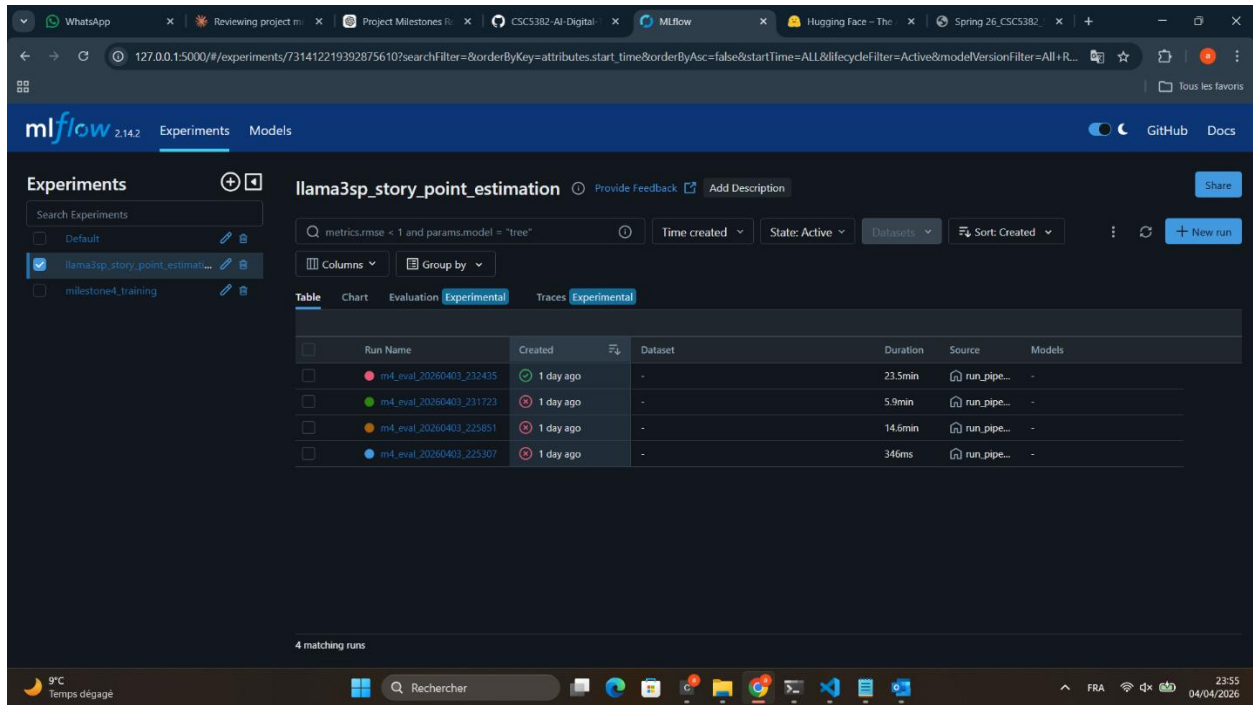
Combined with the deterministic configuration managed through params.yaml and the artifact logging provided by MLflow, Git versioning completes the reproducibility framework: any pipeline run can be reconstructed from a given commit hash and configuration file.

5. Experiment Tracking with MLflow & model versioning

MLflow is used as the primary experiment tracking platform for this milestone. It provides structured logging of parameters, metrics, and artifacts, enabling comparison across pipeline runs and long-term auditability of model evaluations.

5.1 Experiment Structure

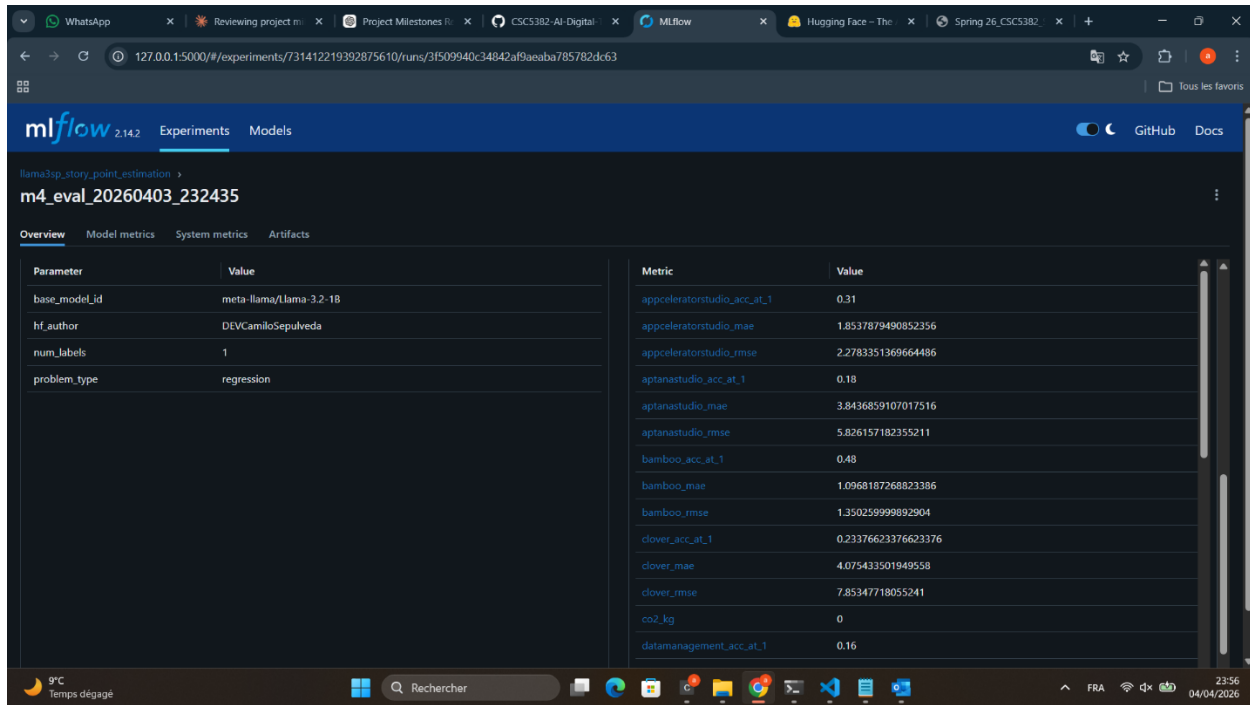
MLflow is configured with a single run per pipeline execution. All per-project evaluation results are logged within this run using a structured naming convention, for example appceleratorstudio_mae, appceleratorstudio_rmse. This design keeps all results from a given execution consolidated in one run while still allowing per-project filtering and comparison.



5.2 Logged Data

The following categories of data are logged to MLflow during each pipeline execution:

Category	Metric / Parameter	Scope
Model Information	base_model_id, hf_author, num_labels, problem_type	Parent run parameters
Per-project Metrics	<project>_mae, <project>_rmse, <project>_acc_at_1	Parent run metrics
Aggregate Metrics	mean_mae, std_mae, mean_rmse, mean_accuracy_at_1, num_projects	Parent run metrics
Carbon / Energy	co2_kg, energy_kwh, inference_time_s	Parent run metrics
Artifacts	mae_per_project.csv, summary.json, carbon_summary.json	Parent run artifacts

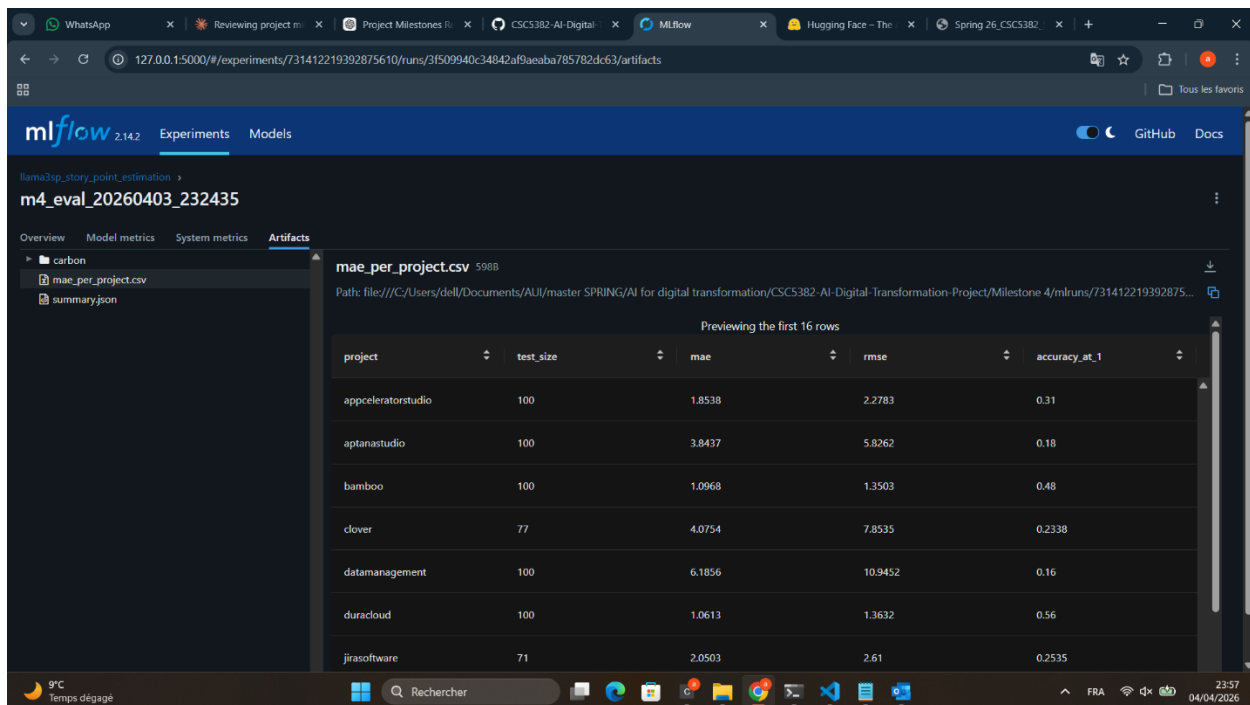


Overview Model metrics System metrics Artifacts

Parameter	Value	Metric	Value
base_model_id	meta-llama/llama-3.2-18	appceleratorstudio_acc_at_1	0.31
hf_author	DEVCamiloSepulveda	appceleratorstudio_mae	1.8537879490852356
num_labels	1	appceleratorstudio_rmse	2.2783351369664486
problem_type	regression	aptanastudio_acc_at_1	0.18
		aptanastudio_mae	3.8436859107017516
		aptanastudio_rmse	5.826157182355211
		bamboo_acc_at_1	0.48
		bamboo_mae	1.0968187268823386
		bamboo_rmse	1.350259999892904
		clover_acc_at_1	0.23376623376623376
		clover_mae	4.075433501949558
		clover_rmse	7.85347718055241
		co2_kg	0
		datamanagement_acc_at_1	0.16

5.3 Model and Artifact Storage

Llama3SP model binaries and LoRA adapters are hosted externally on the HuggingFace Hub, accessed via the HF_TOKEN environment variable at runtime. MLflow stores only metadata, evaluation metrics, and result artifacts locally within the mlruns/ directory. This separation of concerns follows best practice for large model storage in MLOps systems.



Overview Model metrics System metrics Artifacts

mae_per_project.csv 598B

Path: file:///C:/Users/dell/Documents/AIJ/master SPRING/AI for digital transformation/CSC5382-AI-Digital-Transformation-Project/Milestone 4/mlruns/731412219392875610/artifacts

Previewing the first 16 rows

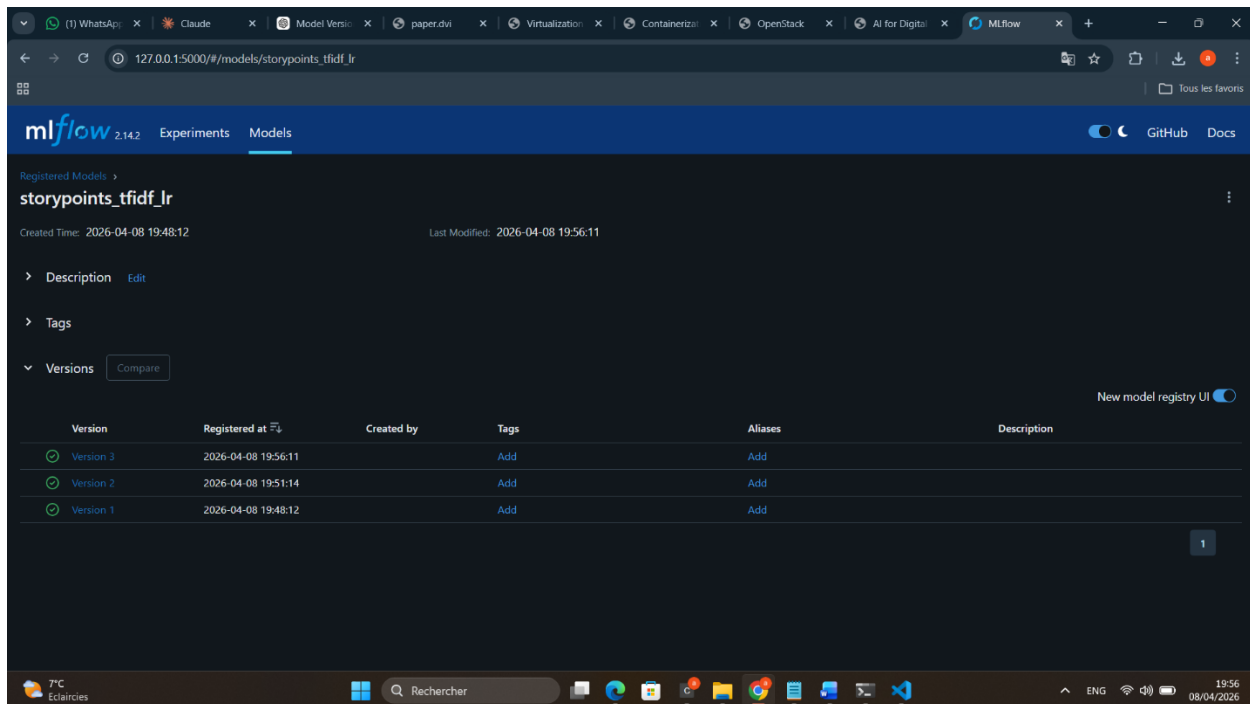
project	test_size	mae	rmse	accuracy_at_1
appceleratorstudio	100	1.8538	2.2783	0.31
aptanastudio	100	3.8437	5.8262	0.18
bamboo	100	1.0968	1.3503	0.48
clover	77	4.0754	7.8535	0.2338
datamanagement	100	6.1856	10.9452	0.16
duracloud	100	1.0613	1.3632	0.56
jirasoftware	71	2.0503	2.61	0.2535

5.4 Viewing Results

The MLflow tracking UI can be launched locally with the command `mlflow ui --port 5000`, after which all runs, metrics, and artifacts are accessible at `http://localhost:5000`. The UI shows run timestamps, aggregate metrics, and an artifact browser for CSV and JSON outputs.

5.5 Model Versioning (MLflow Model Registry)

Beyond experiment tracking, MLflow was extended with Model Registry support to enable proper model versioning and lifecycle management.



MLflow was configured with a SQLite database-backed backend store:

```
mlflow server --backend-store-uri sqlite:///mlflow.db
```

```
--default-artifact-root ./mlartifacts
```

```
--host 127.0.0.1 --port 5000
```

This setup enables persistent tracking of model versions and their associated metadata, separate from the file-based `mlruns/` store.

- Registered Model

A baseline TF-IDF + Linear Regression pipeline was registered in the MLflow Model Registry under the name `storypoints_tfidf_lr`. Each retraining run automatically registers a new version of this model.

- Model Versions

Three versions were created by modifying hyperparameters in configs/params.yaml:

- Version 1: Initial TF-IDF configuration
- Version 2: Increased feature space (max_features)
- Version 3: Extended n-gram range (ngram_range = [1, 2])

- Evaluation from Registry

Models are loaded directly using versioned URIs, for example:

```
python -m src.models.evaluate --model-name storypoints_tfidf_lr --version 1
```

This ensures reproducibility, clear separation between training and evaluation, and the ability to compare model versions systematically.

6. MLOps Platform Integration with ZenML

ZenML is used as the MLOps orchestration platform for this milestone. It provides a structured, reproducible, and trackable pipeline execution environment through its step-based DAG (Directed Acyclic Graph) model.

The screenshot displays the ZenML web interface in a browser. The URL bar shows a project path. The interface includes a navigation bar with 'default' and 'Pipelines' tabs. A specific pipeline, '#0009-milestone4_training_pipeline-2026_04_06-02_16_38_663360', is selected and marked as completed with a green checkmark. Below the pipeline name, there are 'Refresh' and 'Stop' buttons. The main area is divided into 'Visualization' and 'Logs' tabs. The 'Visualization' tab shows a Directed Acyclic Graph (DAG) of the pipeline steps: 'train_tracking_step' (20s) and 'load_model_step' (0s) are at the top, both with green status icons. They both point to an 'output dict' node. These nodes then point to an 'evaluate_step' (52min 47s), which points to another 'output dict' node, which then points to a 'report_step' (0s), and finally to a third 'output dict' node. The 'Run Insights' panel on the right shows details for the pipeline: Id (f6139b72-dba1-431e-96cb-892c89f95c85), Status (completed with a green checkmark), Pipeline (milestone4_training_pipeline), and other fields like 'Replay of', 'Snapshot', and 'Deployment' are marked as 'Not available'.

6.1 Pipeline Architecture

The pipeline is structured as a four-step sequential DAG, with each step calling the appropriate modular components from the `src/` packages. The execution flow is as follows:

- `train_tracking_step`: Trains the lightweight TF-IDF + Linear Regression baseline model and logs the corresponding metrics and artifacts to MLflow. This step demonstrates the training capability of the pipeline.
- `load_model_step`: Resolves the base model identifier from the HuggingFace adapter configuration, then loads the tokenizer and Llama 3.2-1B base model onto CPU memory. The loaded model object is passed to subsequent steps.
- `evaluate_step`: The primary evaluation step. For each of the 16 JIRA projects, it dynamically loads the project-specific LoRA adapter, runs batched CPU inference on the test split, computes MAE, RMSE, and Accuracy@+1, logs all results to MLflow under the active run, and saves a per-project predictions CSV. CodeCarbon wraps the entire evaluation loop to measure inference emissions.
- `report_step`: Aggregates per-project metrics, prints a leaderboard sorted by MAE to standard output, and returns the complete summary dictionary.

6.2 Implementation Details

Steps are defined using the ZenML `@step` decorator in `zenml_steps.py` and composed into the pipeline using the `@pipeline` decorator in `zenml_pipeline.py`. The pipeline entry point is `run_pipeline.py`, which initializes the ZenML tracking context and triggers execution.

A key engineering decision in the `evaluate_step` is the use of dynamic LoRA adapter switching: rather than reloading the base model for each of the 16 projects, the adapter weights are swapped in memory while the base model remains loaded. This significantly reduces the total inference time on CPU-constrained hardware.

6.3 Execution Performance

The complete pipeline was executed on a CPU-only machine (Intel Core i7-1165G7) without GPU acceleration. The total pipeline runtime was 53 minutes and 14 seconds across all 16 projects, with approximately 52 minutes attributed to the `evaluate_step` alone. ZenML step timing is tracked automatically and visible in the ZenML dashboard.

```

C:\Windows\System32\cmd.exe
[Step 3] Aggregate metrics: {'mean_mae': 3.0927, 'std_mae': 2.5198, 'mean_rmse': 4.4558, 'mean_accuracy_at_1': 0.308, 'num_projects': 16}
Step evaluate_step has finished in 52m47s.
Step report_step has started.
[Step 4] Generating final evaluation report...

=====
Milestone 4 – Evaluation Leaderboard (sorted by MAE ↑)
=====
Project      n      MAE      RMSE      Acc@±1
-----
duracloud    100    1.0613    1.3632    0.560
bamboo       100    1.0968    1.3503    0.480
talendesb    100    1.1015    1.4136    0.580
mesos        100    1.3783    1.9684    0.450
usergrid     97     1.5067    1.8958    0.361
springxd     100    1.6722    2.1395    0.370
appceleratorstudio 100    1.8538    2.2783    0.310
jirasoftware  71     2.0503    2.6100    0.254
mule         100    2.4297    2.9878    0.290
titanium     100    2.8328    3.6872    0.260
mulestudio   100    3.4888    4.7076    0.160
talenddataquality 100    3.6042    5.1927    0.210
aptanastudio 100    3.8437    5.8262    0.180
clover       77     4.0754    7.8535    0.234
datamanagement 100    6.1856    10.9452   0.160
moodle       100    11.3025   15.1241   0.070
=====

Macro-average – MAE: 3.0927 ± 2.6024 | RMSE: 4.4558 | Acc@±1: 0.308
=====
Step report_step has finished in 0.914s.
Pipeline run has finished in 53m14s.

=====
Pipeline complete!
View results: results/mae_per_project.csv
MLflow UI:    mlflow ui --port 5000
              http://localhost:5000
=====

C:\Users\dell\Documents\AI\master SPRING\AI for digital transformation\CSC5382-AI-Digital-Transformation-Project\Milestone 4>

```

7. Energy Efficiency and Environmental Monitoring

As an optional component, CodeCarbon is integrated into the evaluation pipeline to measure and report the environmental impact of model inference. This reflects growing awareness in the machine learning community of the computational and carbon costs associated with large-scale model evaluation.

7.1 Integration Approach

The CodeCarbon tracker wraps the full evaluation loop within the `evaluate_step`. It operates in offline mode, using the electricity carbon intensity factor for Morocco (MAR) to estimate CO₂ equivalent emissions. The tracker records energy consumption in kilowatt-hours, estimated CO₂ emissions in kilograms, and total execution duration.

7.2 Outputs

Environmental monitoring results are stored in two files generated at runtime: `results/carbon/emissions.csv`, which contains the full CodeCarbon output, and `results/carbon/carbon_summary.json`, which provides a concise summary formatted for MLflow logging. Both the energy and emissions metrics are logged to MLflow alongside the evaluation results, enabling end-to-end traceability of the computational cost of each pipeline run.

8. Evaluation Results

The following results correspond to the offline evaluation of the pretrained Llama3SP model across all 16 JIRA projects, comprising a total of 1,441 test issues. Results are also stored in results/mae_per_project.csv and logged to MLflow experiment 731412219392875610.

8.1 Per-Project Performance

Project	n	MAE	RMSE	Acc@±1
duracloud	100	1.0613	1.3632	0.560
bamboo	100	1.0968	1.3503	0.480
talendesb	100	1.1015	1.4136	0.580
mesos	100	1.3783	1.9684	0.450
usergrid	97	1.5067	1.8958	0.361
springxd	100	1.6722	2.1395	0.370
appceleratorstudio	100	1.8538	2.2783	0.310
jirasoftware	71	2.0503	2.6100	0.254
mule	100	2.4297	2.9878	0.290
titanium	100	2.8328	3.6872	0.260
mulestudio	100	3.4888	4.7076	0.160
talenddataquality	100	3.6042	5.1427	0.210
aptanastudio	100	3.8437	5.8262	0.180
clover	77	4.0754	7.8535	0.234
datamanagement	100	6.1856	10.9452	0.160
moodle	100	11.3025	15.1241	0.070
Macro-average	1,441	3.0927 ± 2.60	4.4558	0.308

8.2 Analysis / results interpretation

Performance varies substantially across projects, with MAE ranging from approximately 1.06 (duracloud) to 11.30 (moodle). This heterogeneity reflects the inherent differences in story point distributions across JIRA projects: projects with smaller, more consistent story point scales tend to yield lower error rates, while projects with wider or more irregular distributions present greater challenges for the model.

The macro-average MAE of 3.09 ± 2.60 story points and Accuracy@ ± 1 of 0.308 indicate that the model correctly predicts the story point value within one unit for approximately 30.8% of issues across all projects. This is consistent with results reported in the Llama3SP literature for CPU-evaluated settings.

It is important to note that these metrics are produced by a pretrained model evaluated without fine-tuning in this pipeline. The primary purpose of the evaluation is to validate the MLOps infrastructure rather than to optimize predictive performance.

9. Requirements Coverage Summary

Requirement	Tool / Approach	Status	Points
Project Structure & Modularity	Cookiecutter layout (src/, configs/, tests/, results/)	Implemented	2
Code Versioning	Git + GitHub, milestone-based branching	Implemented	2
Experiment Tracking & Model Versioning	MLflow – per-project metrics, artifacts, aggregate stats + Model Registry with 3 registered versions of storypoints_tfidf_lr	Fully Working	5
MLOps Platform Integration	ZenML @step/@pipeline; 4-step DAG executed in 53m14s	Fully Working	5
Energy Efficiency (Optional)	CodeCarbon with Morocco carbon intensity, MLflow logging	Fully Working	+2

10. Conclusion

Milestone 4 demonstrates how a production-grade machine learning system is structured and operated in practice. Rather than focusing exclusively on model performance, this milestone prioritizes the infrastructure surrounding the model: reproducible pipelines, structured experiment tracking, modular code organization, and environmental monitoring.

The four-step ZenML pipeline successfully orchestrates the full evaluation workflow across 16 real-world JIRA projects, with all metrics and artifacts systematically logged to MLflow. The MLflow Model Registry further extends this by versioning the baseline model across multiple configurations, enabling reproducible evaluation and traceability between hyperparameter changes and outcomes. The CodeCarbon integration

adds an environmental accountability layer that aligns with growing expectations for sustainable AI development.

The design philosophy of this milestone reusing pretrained models within a well-engineered evaluation framework reflects contemporary MLOps practice and provides a solid foundation for future work, including online fine-tuning, hyperparameter optimization, and deployment integration.

References

- [1] Sepúlveda Montoya, C. et al. (2025). Llama3SP: Resource-efficient LLM for Agile story point estimation. GitHub: DEVCamiloSepulveda/llama3sp.
- [2] MLflow Documentation. <https://mlflow.org/docs/latest/index.html>
- [3] ZenML Documentation. <https://docs.zenml.io>
- [4] CodeCarbon Documentation. <https://mlco2.github.io/codecarbon/>
- [5] Cookiecutter Data Science. <https://drivendata.github.io/cookiecutter-data-science/>